1 Real-time engine

The real-time path reuses the same estimator, measurement model and state manager as post-processing, but reorganises the control flow around a *streaming* contract: complete RTCM 3 (or SBF) frames arrive one at a time over a network socket, are decoded into observations / corrections / broadcast ephemeris, and a buffered epoch is pushed through the Kalman-filter predict/update cycle the instant its boundary is detected. Where post-processing iterates a closed list of epochs in time order with the entire orbit / clock product set already on disk, the engine sees each epoch exactly once, with corrections that may lead or lag the observations, and can never look ahead. The entry point is `Application::runRealTime` (`src/core/app/ApplicationRealtime.cpp`), which constructs a `RealTimeEngine` (`src/core/realtime/RealTimeEngine.cpp`) and drives it from a single-threaded multi-source poll loop. This section documents the parts that are *specific* to that path; the estimation mathematics, observation equations and SSR correction model are documented elsewhere and only cross-referenced here.

1.1 Streaming control flow

The poll loop holds one `RealtimeStream` per logical source and services them round-robin without ever blocking on a single one:

- a *primary* stream carrying rover observations (RTCM 3 MSM or Septentrio SBF, from a file replay, an NTRIP mountpoint, or a raw TCP socket);
- an optional *corrections* stream (typically an IGS-RTS NTRIP mountpoint carrying SSR);
- an optional *ephemeris* stream (broadcast nav for an obs-only station that does not co-broadcast 1019/1020/...).

Each iteration drains a bounded burst from every active source through its framer and dispatches complete frames to the engine; the primary obs drain is byte-capped per iteration so a high-rate observation stream cannot starve the sparse-but-critical SSR / nav streams (`pumpFrames`). The loop exits when every source goes inactive (file EOF or socket teardown), when a cooperative stop is requested (Ctrl+C / console-close handler), or when the `processing.max_duration_s` data-time budget latches; on exit the final buffered epoch is flushed before teardown so no observations or trace rows are lost.

A socket source is treated as *active* while its client intends to run, not merely while momentarily connected, so a transient drop or the brief pre-connect window does not end the session prematurely; the client reconnects underneath.

1.2 NTRIP transport

Caster connection. `NtripClient` (`src/core/realtime/NtripClient.cpp`) is an NTRIP v2 client built on libcurl. A background thread holds the HTTP GET open and pushes received bytes into a mutex-guarded ring buffer with a hard cap (4 MiB); the buffer applies back-pressure by dropping the oldest bytes when full. Consumers pull through a blocking `read` or a timeout-bounded `tryRead` (the latter is what the multi-stream loop uses so one stream cannot block the others). The request carries the NTRIP v2 headers (`Ntrip-Version:Ntrip/2.0`, a `User-Agent`); the mountpoint is the URL path; HTTP Basic authentication (`CURLAUTH_BASIC`) is supplied when a username is configured, and an optional NMEA GGA sentence is POSTed for VRS mountpoints.

Transport security. The transport defaults to plain HTTP. Plain-HTTP casters (IGS-IP, `igs-ip.net`, SBF replay) work out of the box, byte-identically. A `use_tls` flag selects the `https` URL scheme and forces `CURLUSESSL_ALL`, which is required for port-443 casters such as CDDIS that offer no plain-HTTP endpoint. Certificate verification (`SSL_VERIFYPEER/VERIFYHOST`) stays on unless a testing-only `tls_insecure` override disables it for self-signed or mis-chained casters. Both flags default off; a GNSS-only / IGS-IP run is unaffected.

Reconnection. The receive loop reconnects automatically. After any transfer ends it classifies the outcome — transport error, a genuine non-200 HTTP status (with actionable hints for 401 / 403 / 404), a zero-byte idle mountpoint, or a clean stream end — logs it, and always backs off for `reconnect_delay_s` before retrying, so a misconfigured mountpoint surfaces as an error rather than spinning invisibly. A non-200 *ICY* response (NTRIP v1 casters answer "ICY 200 OK", reported by libcurl as code 0) is treated as success.

1.3 RTCM 3 framing and CRC

`RtcmStreamReader` (`src/core/realtime/RtcmStreamReader.cpp`) turns an arbitrary byte stream into discrete RTCM 3 frames, decoupling transport from parsing. An RTCM 3 frame is

$$\underbrace{\texttt{0xD3}}_{\text{preamble}}\ \underbrace{[\,6\text{ reserved}\,|\,10\text{-bit length }L\,]}_{2\text{ bytes}}\ \underbrace{[\,\text{payload, }L\text{ bytes}\,]}\ \underbrace{[\,\text{CRC-24Q}\,]}_{3\text{ bytes}}, \tag{1}$$

so the total frame length is $3 + L + 3$. `nextFrame` scans for the `0xD3` preamble (discarding non-preamble bytes), reads the 10-bit payload length, waits until the full $3 + L + 3$ bytes are buffered, and verifies the trailing 24-bit Qualcomm CRC (CRC-24Q, table-driven) computed over header+payload. A CRC mismatch advances past the candidate preamble byte and keeps scanning — a corrupt or misframed byte never wedges the reader. Only CRC-valid frames are emitted, so the downstream parser (`RtcmParser`) may assume frame integrity. The reader tracks frames-extracted and bytes-discarded counters for stream-health diagnostics.

Septentrio SBF. A parallel binary path decodes Septentrio SBF frames into the same observation / ephemeris outputs (`src/core/sbf/`): `SbfStreamReader` does the `$@` (`0x24 0x40`) sync and CRC-16/XMODEM check, `SbfParser` dispatches blocks by ID, `Meas3Decoder` unpacks the `MeasEpoch` / `Meas3Ranges` observations (GLONASS FDMA slots resolved through `SbfSignalMap`), and `SubframeAssembler` rebuilds broadcast ephemeris from raw-nav subframes. Only the framing and feed seam (Sections 1.1 and 1.9) are covered here. The `Meas3Ranges` differential range reconstruction — the direct analogue of (2) and (3) below, and the one part of the SBF path that is report-worthy algebra rather than ICD bookkeeping — is *not* yet derived in this reference; it is a known gap, not material documented elsewhere. The remaining SBF detail (`SubframeAssembler` ICD scaling, the `SbfSignalMap` tables, the framing constants) belongs in the Doxygen headers and is deliberately out of scope here.

The top-level `RtcmParser` (`src/core/rtcm/RtcmParser.cpp`) reads the 12-bit message type DF002 from the *first 12 bits of the payload* (not the header) and dispatches: 1005/1006 station coordinates, 1033 receiver/antenna descriptor and 1230 GLONASS code-phase biases (both surfaced-not-consumed), 4076 IGS-SSR, legacy SSR (1057–1068, 1240–1270), broadcast ephemeris (1019/1020/1042/1044/1046), and MSM4 / MSM5 / MSM7 observations. The week needed to promote the various truncated epoch fields to a full `GnssTime` comes from the parser's per-stream time tracker, not from wall-clock at each frame (Section 1.6).

1.4 MSM observation decode

MSM observation messages are decoded by a single body (`decodeMsmImpl`, `src/core/rtcm/RtcmMsmDecoder.cpp`) parameterised over an `MsmVariant` descriptor that supplies the per-field bit widths, scale factors, sentinels and capability flags. Three variants are emitted, MSM4, MSM5 and MSM7 (MSM6 is described by the same variant mechanism, but no variant entry is instantiated for it, so it is not currently produced). The header carries a 64-bit satellite mask, a 32-bit signal mask, and an $N_{\text{sat}} \times N_{\text{sig}}$ cell mask; the number of cells is $N_{\text{cell}} = \text{popcount(cell mask)}$. Each field block is packed contiguously across all satellites or all cells, in mask order, and decoded into parallel vectors before recombination.

Range reconstruction. For each satellite the rough range in milliseconds is assembled from the 8-bit integer (DF397) and the 10-bit modulo (2^{-10} ms, DF398) parts, and each cell adds a signed fine term scaled per variant. Writing c for the speed of light, the pseudorange and carrier phase for a cell are

$$P = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{P}_{\text{fine}}\, \sigma_P\Big), \tag{2}$$

$$\Phi_{[\text{m}]} = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{\Phi}_{\text{fine}}\, \sigma_\Phi\Big), \qquad \Phi_{[\text{cyc}]} = \Phi_{[\text{m}]}/\lambda, \tag{3}$$

with fine scales $\sigma_P = 2^{-24}$, $\sigma_\Phi = 2^{-29}$ for MSM4 and $\sigma_P = 2^{-29}$, $\sigma_\Phi = 2^{-31}$ for MSM7 (all in ms). RTCM-defined sentinel values mark invalid cells and are skipped. Carrier phase is emitted in cycles, matching the RINEX-3 convention consumed by the observation model (the observation-equations section); when the wavelength λ cannot be resolved the phase is carried in metres as a fallback. CNR (DF408/DF403) and, for MSM5/7, Doppler from the combined rough+fine phase-range-rate are also produced; the half-cycle-ambiguity bit sets the LLI half-cycle flag.

DF419 extended-satellite-info gating. A subtle framing hazard governs the per-satellite block. The 4-bit DF419 "extended satellite information" field is present in the bitstream for *every* constellation in MSM5/MSM7, not only GLONASS; only GLONASS *assigns it meaning* (the FDMA frequency-channel number). The decode therefore separates two concerns:

- *bitstream consumption* is gated on the MSM variant (`honour_ext_sat_info`, true for MSM5/MSM7, false for MSM4/MSM6) — when set, $4\,N_{\text{sat}}$ bits are read for all systems;
- *interpretation* is gated separately on the system descriptor; only GLONASS maps the value to a channel.

This is the fix for the GPS L1/L2 geometry-free sign anomaly: reading DF419 only for GLONASS (or omitting it for MSM7) under-consumes the per-satellite block by $4\,N_{\text{sat}}$ bits, so the per-cell fine pseudoranges are read from the wrong bit offset. The corruption preserves the rough range but biases the fine terms, producing an L1/L2 split of order hundreds of metres with the wrong sign on the geometry-free combination — plausible-looking but wrong observations that survive a naive over-read check. The gate matches RTCM 10403.3 and the cssrlib reference. A complementary framing guard in the parser (`warnIfUnderConsumed`) warns whenever a decode leaves ≥ 8 unread payload bits, catching exactly this class of silent misalignment that the bit reader's over-read flag cannot.

1.5 Broadcast nav, SSR and station coordinates

Broadcast ephemeris. Messages 1019 (GPS LNAV), 1044 (QZSS), 1046 (Galileo I/NAV), 1042 (BeiDou D1) and 1020 (GLONASS) are decoded by `RtcmNavDecoder` into a common `BroadcastEphemeris` and pushed into the process-wide `LiveEphemerisStore`, which is what lets the geometry layer resolve satellite orbits with no SP3/CLK file loaded. The per-message field order differs by constellation (e.g. QZSS leads with its clock block); GLONASS carries a sign-magnitude PZ-90 state integrated separately.

SSR corrections. Both the legacy SSR ranges and the IGS-SSR 4076 container are decoded by `RtcmSsrDecoder` and pushed into the shared `SsrCorrectionStore` carrying the real SSR header (provider epoch and IOD-SSR) so staleness and IOD diagnostics reflect the provider solution rather than wall-clock receipt. Orbit, clock, combined, code-bias, phase-bias, URA and high-rate-clock records are stored per satellite; the orbit / clock query paths consult the store transparently, so no per-epoch wiring is needed. The correction *mathematics* — the radial/along/cross orbit projection, the clock and bias application, the iono model — is documented in the SSR-corrections section and is not repeated here. Two real-time-specific reactions to the stream are worth noting:

- an incremented IGS-SSR phase-bias *discontinuity counter* for a signal breaks the integer property of that signal's ambiguity, so the engine resets the affected ambiguity before the next update (`StateManager::noteSsrDiscontinuity`);
- every SSR push re-attempts a held observation epoch (Section 1.7), since the freshest correction horizon has just advanced.

Station coordinates (1005/1006). `decodeStationCoords` reads the antenna reference point (ARP) as three signed 38-bit ECEF components in 0.1 mm units; 1006 additionally carries a 16-bit antenna height. For a reference station that streams its own coordinates this provides an a-priori seed (Section 1.9).

1.6 Time-system handling

Streaming messages carry truncated, system-specific epoch fields that must be promoted to a common GPS-time `GnssTime`. Each parser instance keeps a continuous-GPS-seconds tracker of the last decoded stream epoch and promotes every mod-week / time-of-day field by *nearest continuation* to that tracker — the week (or day) minimising $|\Delta t|$, RTKLIB's `adjweek` semantics (`RtcmParser::anchorGpsSeconds`). The host clock (`TimeUtils::nowContinuousGpsSeconds`, POSIX UTC plus the leap offset) is consulted *only* at cold start, on the first time-bearing frame after construction, so a skewed host clock or a promotion race at a week rollover cannot re-week an established stream. Each successful MSM/SSR epoch advances the tracker (`updateTimeAnchor`); the truncated broadcast-nav weeks only read it. The anchor below is this tracker's current value.

- **Flat ms-of-week systems** (GPS/GAL/QZS/SBAS/BDS MSM): the 30-bit epoch is milliseconds-of-week; BeiDou adds the constant 14 s BDT→GPS offset (with a week-rollover guard).
- **GLONASS** carries its 30-bit MSM epoch as a 3-bit day-of-week plus a 27-bit time-of-day in ms — not 27 bits total; reading it short misaligns the satellite mask and corrupts the whole GLONASS decode. The ms-of-day is promoted by `TimeUtils::glonassTodToGpsTime`, which carries the reported time-of-day through the GLONASS = UTC(SU)+3 h ("Moscow") day frame and back to GPS time, anchoring on the stream-time tracker and re-adding leap seconds:

$$t_{\mathrm{GPS}} = \left(t^{0}_{\mathrm{day}} + \tilde{t}_{\mathrm{TOD}} \right) - \Delta_{\mathrm{Moscow}} + \Delta_{\mathrm{leap}}, \tag{4}$$

where t^0_{day} is the Moscow-day midnight nearest the anchor and $\tilde{t}_{\text{TOD}}$ is the reported time-of-day placed into that day by resolving the midnight wrap toward the nearest match ($\pm 12\,\text{h}$). The redundant 3-bit day-of-week is left unread. The same conversion promotes the 1020 ephemeris `toe` and any GLONASS SSR epoch, so observations, orbits and corrections share one clock.

- **GLONASS FDMA wavelength.** GLONASS L1/L2 are frequency-division: each slot transmits on $f = f_{\text{base}} + k\,\Delta f$ with channel $k \in [-7, +6]$, so the carrier wavelength is per-satellite. The channel is decoded from DF419 as $k = v - 7$ for the valid range and used in Equation (3); it is also carried on the observation so the slot map and MW combinations can resolve frequency. When DF419 is absent (MSM4/MSM6) or reserved the phase is carried in metres rather than fabricating a bogus $k = -7$.

Each decoded record stamps its own content epoch (`epoch_time`), which is what the synchronisation logic of Section 1.7 compares against, rather than wall-clock arrival.

1.7 Observation/SSR synchronisation

PPP-RTK needs observations time-*aligned* with the corrections, not pinned to wall-clock. In a live session the obs stream commonly runs ahead of a lagging SSR stream; processing an obs epoch that no satellite's SSR can cover would drive the whole epoch broadcast-grade (and, with `drop_broadcast_grade`, wipe it). The opt-in `processing.enable_obs_ssr_sync` hold buffer guards against this. Let t_{obs} be the obs epoch and define the SSR coverage horizon from the freshest stored SSR content epoch (`SsrCorrectionStore::newestSsrEpoch`) plus the maximum SSR age:

$$t_{\text{horizon}} = t^{\text{newest}}_{\text{SSR}} + \tau_{\max}, \qquad \tau_{\max} = \texttt{ssr_max_age_s}. \tag{5}$$

The decision is:

- $t_{\text{obs}} \le t_{\text{horizon}}$: SSR covers the epoch — process normally and clear any hold;
- $t_{\text{obs}} > t_{\text{horizon}}$: *defer* the epoch. The held buffer keeps the latest such epoch only (skip-to-latest semantics), and `drainHeldObs` re-attempts it the moment an SSR push advances the horizon.

A liveness backstop prevents a dead SSR feed from starving the filter forever: if a hold persists past `obs_ssr_sync_max_hold_s` the engine latches a degraded "outage" and passes every epoch through (broadcast-grade / dropped via the existing path) until SSR re-covers the obs. The held epoch is drained with skip-to-latest *bypassed* so the intended SSR-aligned target is not discarded, and the final `flush` bypasses both the hold and skip-to-latest so the last epoch is never lost. The whole feature is inert when disabled or in any non-live-SSR run.

A separate, coarser `enable_obs_skip_to_latest` drops obs epochs whose lag behind *wall-clock* GPS time exceeds a bound, so a backlogged stream fast-forwards to the freshest epoch; the skip returns before any continuity state is touched, so the resume correctly trips the gap path (Section 1.10).

1.8 Ephemeris management

`LiveEphemerisStore` retains, per satellite, the current record and one previous record, and resolves the orbit/clock against whichever is appropriate. Two policies matter in real time.

- **Freshest-toe selection.** An incoming record is rejected when its `toe` (expressed in GPS-continuous seconds, `toeGpsContinuous`, so BDT/GLONASS scales are reconciled) is older than the held record beyond a sub-second epsilon, so a stale or re-ordered broadcast never overwrites a

newer one. An authoritative IGS-stream record is additionally protected from being clobbered by a receiver-decoded record at equal `toe`: receiver nav is accepted only as a genuine update or as a fallback once the IGS stream has stalled past a threshold (the GLONASS nominal `toe` step).

- **toe-aware / IOD-aware pivoting.** For orbit and clock evaluation the store picks the record whose `toe` is nearest the query epoch (`pick`); for an SSR-corrected satellite it first re-references to the record the SSR delta was computed for (`pickForSsr`: match the SSR IODE against the current or retained-previous record). Across an ephemeris cutover the receiver can be one IOD ahead of the SSR stream; without this re-reference the IOD mismatch would needlessly drop the satellite to broadcast-grade.

1.9 A-priori seeding from RTCM 1005/1006

A *live* SBF/RTCM observation stream carries no RINEX `APPROX POSITION` record (a file-fed source does; see the third bullet below). Without a finite a-priori the first `computeModels` rejects every satellite on a position-sanity check (almost all sit below the elevation mask when the seed is the default Earth-centred fallback at the geographic pole), so WLS never runs and the filter cannot bootstrap. The engine therefore seeds the filter a-priori from the stream itself until the Kalman filter takes ownership of the position:

- from a *PVTGeodetic* solution in an SBF stream (`feedSbfFrame`); and
- from an RTCM *1005/1006* station ARP (`feedRtcmFrame`), the RTCM analogue of the PVT seed, which makes `receiver.reference_position` optional for any station streaming 1005/1006; and
- from the RINEX header of a *file-fed* source, reduced from the marker to the antenna reference point through `ANTENNA:DELTA H/E/N` (`RinexParser::getArpPositionXyz`, over `Coordinates::markerToArp`). The ARP, not the monument, is the frame the position state and the ATX PCO/PCV applied to it live in, so seeding on `APPROX POSITION XYZ` verbatim would start the filter one eccentricity out of frame. The reduction is an exact no-op on a zero or absent eccentricity, so a live SBF/RTCM stream is unaffected; it is observable only for the file-fed closed-loop consumer, whose header carries a real eccentricity. There is no double application — `PppModel::initializeState` reduces from the *raw* marker, and only while the state position is still zero or non-finite.

A configured `reference_position` always wins. The seed is applied only while `first_epoch_` holds; the startup-defer gate keeps that flag set through warmup, so a 1005/1006 message arriving a few seconds into the stream (these come less often than the 1 Hz obs) still seeds before the first estimation epoch. Once the first epoch is fully processed the flag clears and the filter owns the position — the ARP / PVT is thereafter only logged for comparison, not re-applied.

1.10 The per-epoch real-time update

`processEpoch` runs the same predict/update pipeline as post-processing, with streaming-specific guards layered on the front and the post-processing-only stages removed. In order:

1. **Lag / sync / duration guards.** The wall-clock backlog warning, the obs↔SSR hold of Section 1.7, the skip-to-latest drop, and the `max_duration_s` latch all run before any continuity state is touched.
2. **Step / gap detection.** $\Delta t = t_k - t_{k-1}$ (with a week-rollover correction). A non-advancing $\Delta t \leq 0$ is a duplicate (a co-timestamped second SBF block or a retransmit) and is skipped cleanly

— folding it in would double-count the measurement and feed $\Delta t = 0$ into the process-noise build, which hard-throws. A Δt beyond the gap tolerance flags a gap.

3. **Model + startup gate.** Observations are filtered to the active systems and run through `computeModels`. When `defer_until_corrections_ready` is armed, estimation is deferred (no predict/update) until a per-constellation cm-grade quorum of broadcast+SSR satellites is present (or a timeout forces a degraded start), so the filter seeds code and phase together on a clean state instead of cold-starting on metre-grade broadcast clocks.
4. **Clock / gauge seeding.** In modes with a receiver-clock state a WLS clock estimate re-seeds the filter clock on a jump, gap, or first epoch. Under BSSD (no receiver-clock state) a single per-epoch median-of-code-innovations receiver clock, common to all satellites, is subtracted from every code and phase observable — an exactly SD-invariant re-gauge that keeps the per-satellite bias states at DCB scale rather than absorbing the full unsteered receiver clock.
5. **Predict.** `predictAided` over $(t_{k-1}, t_k]$ — identical to the shared aiding path; for a GNSS-only stream it delegates byte-identically to `predict`(Δt) of the estimation-engine section.
6. **Cycle-slip detection / repair.** The MW detector plus the optional TurboEdit MW+GF joint repair and soft-LLI gap bridge run pre-update, sharing the same detectors and repair machinery as the cycle-slip section. These are enabled under `estimation.cycle_slip`; TurboEdit is high-rate-only and seeds its GF-arc gap tolerance from the configured stream rate (a live stream has no RINEX interval header).
7. **Measurement update.** The same outlier / forced-pivot-handover rebuild loop as the post-processing driver (shared via `EpochUpdateSupport`): rebuild on `NeedsMeasurementRebuild`, forward each constellation's forced pivot handover, re-inflate reset-satellite states, clear MW/TurboEdit arc history for KF-reset signals, and peel one outlier per pass. An optional soft a-priori ZWD pseudo-observation anchors the BSSD [up, ZWD] near-null subspace.
8. **Ambiguity resolution.** After a successful float update, MLAMBDA AR runs on the committed state with the same configuration surface as post-processing, fed the MW-resolved wide-lane integers (the ambiguity-resolution section).
9. **State extraction + trace.** The posterior is synced back to the PPP state and trace rows are written.

Post-processing stages skipped in real time. The non-causal and batch stages have no place in a streaming, look-ahead-free pipeline and are deliberately absent:

- the backward RTS smoother is force-disabled under `AppMode::RealTime` — smoothing requires the full forward trajectory and cannot run causally;
- geometry relinearisation (the IEKF second pass) is a post-processing-only refinement and is not replicated here;
- precise SP3/CLK products are not loaded when a live broadcast+SSR source supplants them (a precise-product *file* replay still loads them);
- higher-order ionosphere is unsupported in the streaming pipeline (no GIM source) and is warned-and-skipped if enabled.